

Frontend Developer — Easy Guide

Crash course before the presentation · Retail Demand Forecasting (Walmart M5)

In one sentence

I built the **React** app you see in the browser — 9 pages, 62 tests — and it holds no data of its own: it asks our backend and draws the answers.

Frontend vs backend

Frontend = what you see and click, running in the user's browser. **Backend** = the program holding the data, running on a server. They talk over **HTTP**, exchanging **JSON**.

```
BROWSER (React)  --- GET /api/v1/... ---> FastAPI
                  <--- JSON -----
draw chart
```

React in five words

Component — a reusable piece of interface (a card, a badge, a chart). **Props** — inputs you pass into a component. **State** — data a component remembers; when it changes, React redraws. **Hook** — a use... function that gives a component React powers (useState). **JSX** — the HTML-looking syntax inside JavaScript.

SPA — Single Page Application. The browser loads one page and JavaScript swaps the content as you navigate. Faster — but the server must return the app for *any* URL, or refreshing /forecast would 404.

Our stack

Piece	Why
React	build the UI from reusable components
TypeScript	types the API responses — a backend rename becomes a compile error, not a blank chart
Vite	fast build; splits output into chunks
Tailwind	one palette in one config file
Recharts	draws history flowing into forecast
TanStack Query	handles caching, loading and error state
React Router	maps URLs to pages

The 9 pages

Route	What it shows
/	Overview — the 30-second answer
/forecast	history + 28-day forecast for one product
/hierarchy	drill from chain down to item
/insights	what needs attention
/accuracy	measured performance
/validation	replay a window where we know the answer

Route	What it shows
/model	architecture + live model re-run
/methodology	data, limits, what we rejected
/assistant	the AI Forecast Assistant

Two ways to reach the assistant — keep both

	Purpose
Sidebar "AI Assistant" + description	tells a first-time visitor it exists
Floating AI button (bottom-right)	shortcut from any page

The floating button is rendered once from AppShell, shows its label on hover **and on keyboard focus**, has `aria-label="Open AI Assistant"`, and navigates through the router — no page reload.

How data gets to the screen

Component -> hooks.ts (TanStack Query) -> client.ts -> /api/v1/... -> FastAPI

No API address is compiled into the bundle. `VITE_API_BASE_URL` is left unset, so the app uses the same-origin path `/api/v1` and nginx proxies it. A test asserts it never matches `localhost`, an IP or an absolute URL — a hard-coded host is the classic reason a build works locally and dies deployed.

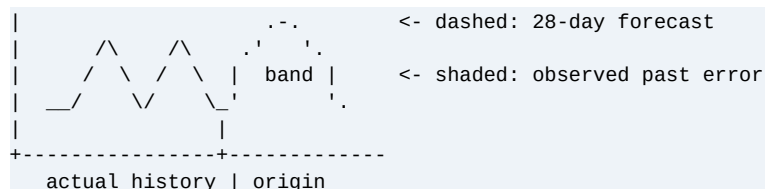
Three states, always

State	User sees
Loading	a spinner with a label
Error	the message, plus a remedy where we have one
Success	the data

Our rule: a failed request must show a visible error, **never** a plausible-looking empty chart. A chart of zeros looks like real data — that's worse than nothing. There's a test for it.

Real bug we hit: the Assistant page had no branch for "status request failed". When an old API returned 404, every section was gated on data that never arrived and the page showed **the heading and nothing else**. Now there's an explicit error branch — and a test asserting the page can never be blank.

The forecast chart



- solid = actual sales · dashed = frozen forecast
- shaded = **observed error, NOT a confidence interval**
- the caption wording comes from the backend's `band_basis` field — we don't

invent it

Production build (verified)

Chunk	Raw	Gzipped
charts	416.32 kB	112.23 kB
react	157.89 kB	51.84 kB
query	47.32 kB	14.73 kB
index	32.45 kB	10.35 kB
Assistant (lazy)	13.07 kB	4.46 kB

Only Overview loads eagerly — every other page is fetched on first visit. That's why the 416 kB chart library isn't on the critical path.

Deep links: nginx uses `try_files $uri $uri/ /index.html`, so opening `/forecast` directly returns the app and React Router shows the right page.

The Assistant page — provenance

Every answer shows:

- **which data** it used (intent)
- **how long** it took (elapsed_ms)
- **whether every number traced back** to our backend (grounded)

If a number can't be traced, we **name it and mark it unverified** — we don't hide it. If the backend refused locally, it says **"answered locally — no AI call"**.

*The frontend has **no Gemini key, no AI SDK, no Gemini endpoint**. It only talks to our backend. A test asserts the DOM contains nothing key-shaped.*

Things we deliberately did NOT build

- **No price what-if slider.** We measured the model's response to simulated price cuts — non-monotone and sometimes backwards. It would have been the flashiest feature in the app. Shipping it would have misled people.
- **No accuracy shown for the delivered forecast** — that window has no ground truth.
- **No fabricated numbers.** Every figure came from an API response.

Tests: 62 across 5 files

`pages.test.tsx · assistant.test.tsx · floating-assistant.test.tsx · client.test.ts · format.test.ts`

Plus TypeScript compiles clean and the production build succeeds in ~2 s.

How to explain it to a judge

"I built the frontend — the React app in the browser. React builds the interface out of reusable components, and we have 9 pages sharing one shell with the header and sidebar.

We use TypeScript, so the API response shapes are typed. If the backend renames a field we get a compile error instead of a silently blank chart.

The frontend holds no data — it asks our FastAPI backend and draws what comes back. And no API address is compiled into the bundle: the app uses a same-origin path and nginx proxies it, so the same build works anywhere.

The main screen shows actual sales as a solid line flowing into the 28-day forecast as a dashed line, with a shaded band. That band is observed past error, not a confidence interval — and the caption says so, using wording that comes from the backend rather than something we made up.

There's also the AI assistant, reachable from the sidebar and a floating button. Every answer shows which data it used and whether every number traced back to our backend. If one can't be traced, we name it and mark it unverified.

And two things we deliberately didn't build: a price what-if slider, because we measured the model's price response and it's backwards; and any accuracy figure for the delivered forecast, because that window has no ground truth."

Quick answers

What is React? A library that builds the UI from reusable components and redraws only what changed. *Where does the data come from?* HTTP requests to our FastAPI backend, as JSON. *Why TypeScript?* API shapes are typed — contract changes fail at compile time. *How do deep links work?* nginx `try_files` returns the app for any path; React Router picks the page. *Is the API host baked in?* No — same-origin `/api/v1`, asserted by a test. *Could the Gemini key leak into the bundle?* No — the frontend has no key and no AI SDK; tested and scanned. *What if the API is down?* An explicit error state with a remedy — never a fake empty chart.

30-second revision

React + TypeScript + Vite + Tailwind + Recharts + TanStack Query + React Router · **9 pages** · AppShell + floating AI button · same-origin `/api/v1`, **no baked host** · lazy loading (Assistant = 13.07 kB chunk) · SPA deep links via `try_files` · loading/error/success always · band = observed error, not a confidence interval · no price slider, no fake numbers · **62 tests**, TS clean, build ~2 s.